UNIVERSITAS DIPONEGORO

Boolean Algebra Part II: Simplification Techniques

VTE2624203 – Elektronika Digital

Dhani Nur Indra Syamputra, S.Si., M.Sc.

SOP & POS Boolean Expression

UNIVERSITAS DIPONEGORO

Sum-of-Product (SOP)

- A sum-of-products expression contains the sum of different terms, with each term being either a single literal or a product of more than one literal.
- It can be obtained from the truth table directly by considering those input combinations that produce a logic '1' at the output.
- Some examples of this form are:

1. $ABC + \overline{A}B\overline{C}$
2. $AB + \overline{A}B\overline{C} + \overline{C}\,\overline{D} + D$
3. $\overline{A}B + C\overline{D} + EF + GK + H\overline{L}$

- Each term in SOP is called **minterm**, the value '0' written in complemented form.

$$Y = \overline{A}\,.\overline{B}.\overline{C} + \overline{A}.B.C + A.B.\overline{C} + A.\overline{B}.C$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

SOP & POS Boolean Expression

UNIVERSITAS DIPONEGORO

Product-of-Sum (POS)

- A product-of-sums expression contains the product of different terms, with each term being either a single literal or a sum of more than one literal.
- It can be obtained from the truth table by considering those input combinations that produce a logic '0' at the output.
- Here are some product-of-sum expressions:

  1. $(A + \overline{B} + C)(A + C)$
  2. $(A + \overline{B})(\overline{C} + D)F$
  3. $(A + C)(B + \overline{D})(\overline{B} + C)(A + \overline{D} + \overline{E})$

- Each term in POS is called **maxterm**, the value '1' is written in complemented form.

$$\boldsymbol{Y = (A + B + \overline{C}) \cdot (A + \overline{B} + C) \cdot (\overline{A} + B + C) \cdot (\overline{A} + \overline{B} + \overline{C})}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Multiplying out the given expression and carrying out the obvious simplification provides the equivalent sum-of-products expression.

$$Y = \overline{A}\,.\overline{B}.\overline{C} + \overline{A}.B.C + A.B.\overline{C} + A.\overline{B}.C$$

SOP -> POS Conversion

UNIVERSITAS DIPONEGORO

taking the dual of the given expression,

multiplying out different terms to get the sum-of-products form,

removing redundancy, and

taking a dual to get the equivalent product-of-sums expression.

$$A.B + \overline{A}.\overline{B}$$

The dual of the given expression $= (A+B).(\overline{A}+\overline{B})$:

$$(A+B).(\overline{A}+\overline{B}) = A.\overline{A} + A.\overline{B} + B.\overline{A} + B.\overline{B} = 0 + A.\overline{B} + B.\overline{A} + 0 = A.\overline{B} + \overline{A}.B$$

The dual of $(A.\overline{B} + \overline{A}.B) = (A+\overline{B}).(\overline{A}+B)$. Therefore

$$A.B + \overline{A}.\overline{B} = (A+\overline{B}).(\overline{A}+B)$$

Canonical Form

UNIVERSITAS DIPONEGORO

A	B	Y	Minterm		Maxterm	
			Term	Symbol	Term	Symbol
0	0	0	$\bar{A}\bar{B}$	m_0	$(A+B)$	M_0
0	1	1	$\bar{A}B$	m_1	$(A+\bar{B})$	M_1
1	0	0	$A\bar{B}$	m_2	$(\bar{A}+B)$	M_2
1	1	1	AB	m_3	$(\bar{A}+\bar{B})$	M_3

- Canonical SOP: combination of input values that produce output '1'

$$Y = \bar{A}B + AB = m_1 + m_3 = \sum m(1,3)$$

- Canonical POS: combination of input vales that produce output '0'

$$Y = (A+B)\cdot(\bar{A}+B) = M_0 \cdot M_2 = \prod M(0,2)$$

Example 1

UNIVERSITAS DIPONEGORO

- State the truth table below in the canonical form of SOP and POS!

	A	B	C	X
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

- SOP Form

$$\begin{aligned} X(A,B,C) &= (\bar{A}\bar{B}\bar{C}) + (\bar{A}BC) + (A\bar{B}\bar{C}) + (AB\bar{C}) + (ABC) \\ &= m_0 + m_3 + m_4 + m_6 + m_7 \\ &= \sum m(0,3,4,6,7) \end{aligned}$$

- POS Form

$$\begin{aligned} X(A,B,C) &= (A+B+\bar{C}) \cdot (A+\bar{B}+C) \cdot (\bar{A}+B+\bar{C}) \\ &= M_1 \cdot M_2 \cdot M_5 \\ &= \prod M(1,2,5) \end{aligned}$$

Standard / Expanded Form

UNIVERSITAS DIPONEGORO

- Expanded sum-of-products and product-of-sums forms of Boolean expressions are useful not only in analyzing these expressions but also in the application of minimization techniques.
- Example: Expand following function into standardized form!
  - $Y = A\bar{B}\bar{C} + \bar{B}C$
  - $Y = AB + \bar{C}$

$$
\begin{aligned}
Y(A,B,C) &= A\bar{B}\bar{C} + \bar{B}C \\
&= A\bar{B}\bar{C} + \bar{B}C(A + \bar{A}) \\
&= A\bar{B}\bar{C} + A\bar{B}C + \bar{A}\bar{B}C \\
&= m_4 + m_5 + m_1
\end{aligned}
$$

$$Y(A,B,C) = \sum m(1,4,5)$$

$$
\begin{aligned}
Y(A,B,C) &= AB + \bar{C} \\
&= AB(C + \bar{C}) + \bar{C}(B + \bar{B})(A + \bar{A}) \\
&= ABC + AB\bar{C} + \bar{C}(AB + \bar{A}B + A\bar{B} + \bar{A}\bar{B}) \\
&= ABC + \underbrace{AB\bar{C} + AB\bar{C}}_{=AB\bar{C}} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} \\
&= ABC + AB\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}
\end{aligned}
$$

$$Y(A,B,C) = \sum m(0,2,4,6,7)$$

Algebraic Simplification

UNIVERSITAS DIPONEGORO

- We can use the Boolean algebra theorems that we studied in previous meeting to help us simplify the expression for a logic circuit.
- Unfortunately, it is not always obvious which theorems should be applied to produce the simplest result.
- Furthermore, there is no easy way to tell whether the simplified expression is in its simplest form or whether it could have been simplified further.
- Thus, algebraic simplification often becomes a process of trial and error.

Simplified Algorithm

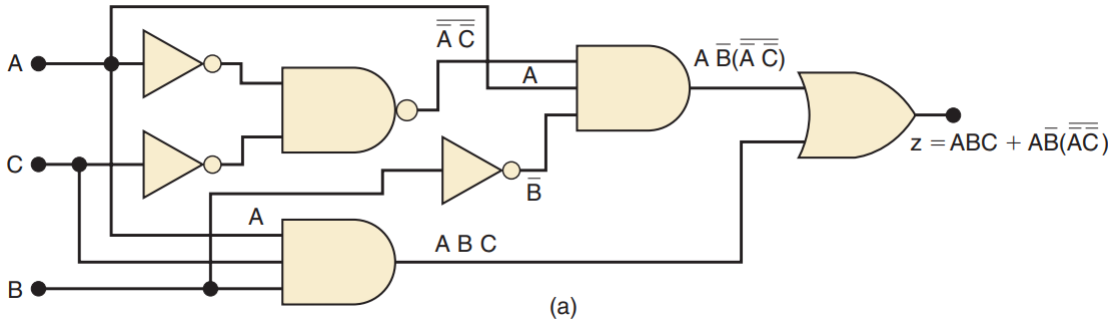
The original expression is put into SOP form by repeated application of DeMorgan's theorems and multiplication of terms.

Once the original expression is in SOP form, the product terms are checked for common factors, and factoring is performed wherever possible. The factoring should result in the elimination of one or more terms.

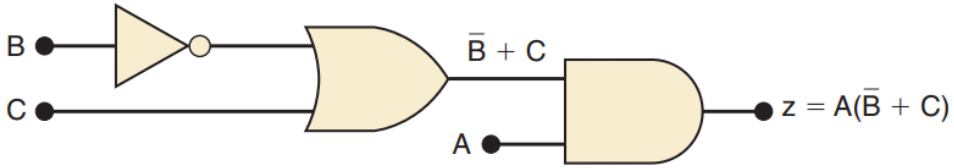
Example 2

UNIVERSITAS DIPONEGORO

- Simplify the following circuit!



(a)



(b)

$$
\begin{aligned}
z &= ABC + A\bar{B}(\overline{\bar{A}\bar{C}}) \\
&= ABC + A\bar{B}(\bar{\bar{A}} + \bar{\bar{C}}) && \text{(DeMorgan's Theorem)} \\
&= ABC + A\bar{B}(A + C) && \text{(Involution Law)} \\
&= ABC + A\bar{B}A + A\bar{B}C && \text{(Distributive Law)} \\
&= ABC + A\bar{B} + A\bar{B}C && \text{(Identitiy Law)} \\
&= ABC + A\bar{B} && \text{(Redundancy Law)} \\
&= AC + A\bar{B} && \text{(Theorem 11)} \\
z &= A(C + \bar{B}) && \text{(Distributive Law)}
\end{aligned}
$$

Karnaugh Map Method

UNIVERSITAS DIPONEGORO

- The Karnaugh map (K map) is a graphical tool used to simplify a logic equation or to convert a truth table to its corresponding logic circuit in a simple, orderly process.
- Although a K map can be used for problems involving any number of input variables, its practical usefulness is limited to five or six variables.

A	B	X
0	0	$1 \rightarrow \bar{A}\bar{B}$
0	1	0
1	0	0
1	1	$1 \rightarrow AB$

$$\left\{ x = \bar{A}\bar{B} + AB \right\}$$

	$\bar{B}$	B
$\bar{A}$	1	0
A	0	1

(a)

A	B	C	X
0	0	0	$1 \rightarrow \bar{A}\bar{B}\bar{C}$
0	0	1	$1 \rightarrow \bar{A}\bar{B}C$
0	1	0	$1 \rightarrow \bar{A}B\bar{C}$
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	$1 \rightarrow AB\bar{C}$
1	1	1	0

$$\left\{ \begin{aligned} X = {} & \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C \\ & + \bar{A}B\bar{C} + AB\bar{C} \end{aligned} \right\}$$

	$\bar{C}$	C
$\bar{A}\bar{B}$	1	1
$\bar{A}B$	1	0
AB	1	0
$A\bar{B}$	0	0

(b)

A	B	C	D	X
0	0	0	0	0
0	0	0	1	$1 \rightarrow \bar{A}\bar{B}\bar{C}D$
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	$1 \rightarrow \bar{A}B\bar{C}D$
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	$1 \rightarrow AB\bar{C}D$
1	1	1	0	0
1	1	1	1	$1 \rightarrow ABCD$

$$\left\{ \begin{aligned} X = {} & \bar{A}\bar{B}\bar{C}D + \bar{A}B\bar{C}D \\ & + AB\bar{C}D + ABCD \end{aligned} \right\}$$

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	0	0
$\bar{A}B$	0	1	0	0
AB	0	1	1	0
$A\bar{B}$	0	0	0	0

(c)

Karnaugh Map Method

UNIVERSITAS DIPONEGORO

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$				
$\overline{A}B$				
AB				
$A\overline{B}$				

AB \ CD	00	01	11	10
00				
01				
11				
10				

Karnaugh Map Method

K-Map: Looping

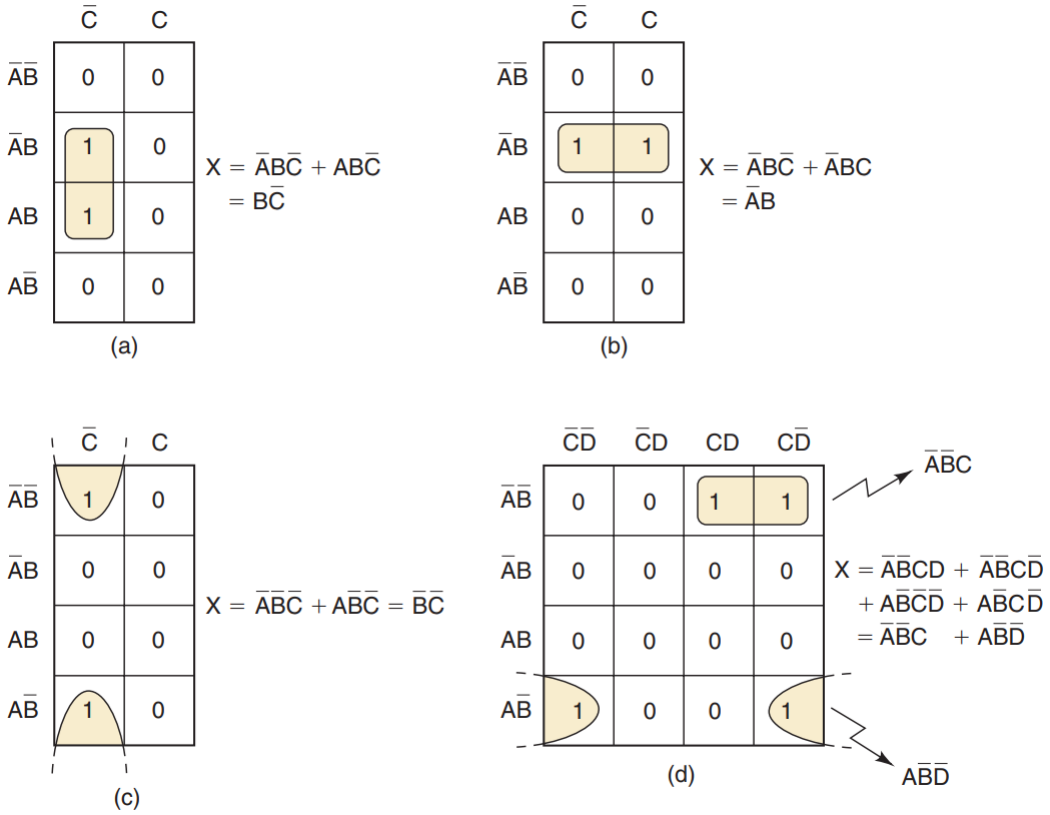
UNIVERSITAS DIPONEGORO

- The expression for output X can be simplified by properly combining those squares in the K map that contain 1s, which is called ***Looping***.

Looping Groups of Two (Pairs)

- *Looping a pair of adjacent 1s in a K map eliminates the variable that appears in complemented and uncomplemented form.*

$$
\begin{aligned}
X &= \bar{A}B\bar{C} + AB\bar{C} \\
&= B\bar{C}(\bar{A} + A) \\
&= B\bar{C}(1) = B\bar{C}
\end{aligned}
$$



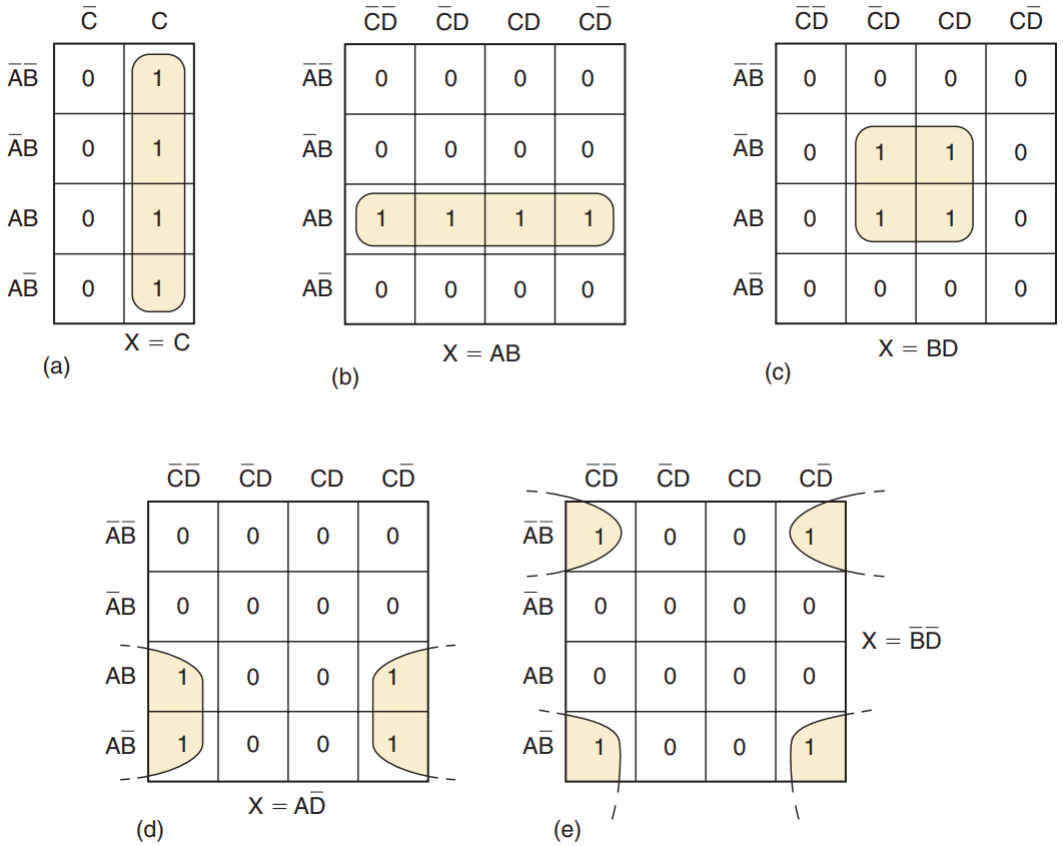
K-Map: Looping Quads

UNIVERSITAS DIPONEGORO

- A K map may contain a group of four 1s that are adjacent to each other. This group is called a ***quad***.

- *Looping a quad of adjacent 1s* ***eliminates the two variables*** *that appear in both complemented and uncomplemented form.*

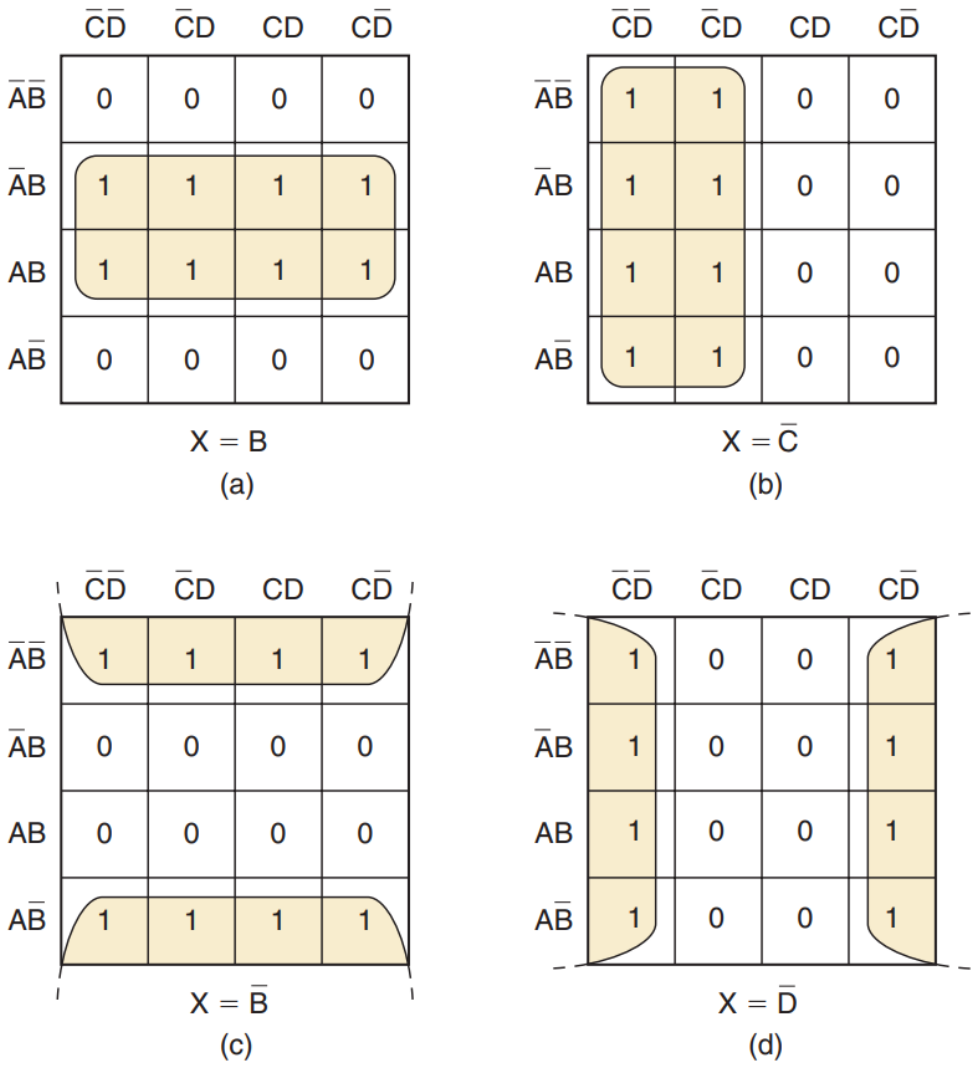
$$
\begin{aligned}
X &= \bar{A}\bar{B}C + \bar{A}BC + ABC + A\bar{B}C \\
&= \bar{A}C(\bar{B} + B) + AC(B + \bar{B}) \\
&= \bar{A}C + AC \\
&= C(\bar{A} + A) = C
\end{aligned}
$$



K-Map: Looping Octets

UNIVERSITAS DIPONEGORO

- A group of eight 1s that are adjacent to one another is called an ***octet***.



Complete Simplification Process

UNIVERSITAS DIPONEGORO

When a variable appears in both complemented and uncomplemented form within a loop, that variable is eliminated from the expression. variables that are the same for all squares of the loop must appear in the final expression.

Step 1
- Construct the K map and place 1s in those squares corresponding to the 1s in the truth table. Place 0s in the other squares.

Step 2
- Examine the map for adjacent 1s and loop those 1s that are not adjacent to any other 1s. These are called *isolated* 1s.

Step 3
- Next, look for those 1s that are adjacent to only one other 1. Loop any pair containing such a 1.

Step 4
- Loop any octet even if it contains some 1s that have already been looped

Step 5
- Loop any quad that contains one or more 1s that have not already been looped, making sure to use the *minimum number of loops*.

Step 6
- Loop any pairs necessary to include any 1s that have not yet been looped, making sure to use the minimum number of loops.

Step 7
- Form the OR sum of all the terms generated by each loop.

Example 3

UNIVERSITAS DIPONEGORO

	$\overline{C}\,\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\,\overline{B}$	0 (1)	0 (2)	0 (3)	1 (4)
$\overline{A}B$	0 (5)	1 (6)	1 (7)	0 (8)
AB	0 (9)	1 (10)	1 (11)	0 (12)
$A\overline{B}$	0 (13)	0 (14)	1 (15)	0 (16)

$$X = \underbrace{\overline{A}\,\overline{B}C\overline{D}}_{\text{loop 4}} + \underbrace{ACD}_{\text{loop 11, 15}} + \underbrace{BD}_{\text{loop 6, 7, 10, 11}}$$

(a)

	$\overline{C}\,\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\,\overline{B}$				
$\overline{A}B$				
AB				
$A\overline{B}$				

Example 4

UNIVERSITAS DIPONEGORO

	$\overline{C}\,\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\,\overline{B}$	0 (1)	0 (2)	1 (3)	0 (4)
$\overline{A}B$	1 (5)	1 (6)	1 (7)	1 (8)
AB	1 (9)	1 (10)	0 (11)	0 (12)
$A\overline{B}$	0 (13)	0 (14)	0 (15)	0 (16)

$$X = \underbrace{\overline{A}B}_{\text{loop 5, 6, 7, 8}} + \underbrace{B\overline{C}}_{\text{loop 5, 6, 9, 10}} + \underbrace{\overline{A}CD}_{\text{loop 3, 7}}$$

(b)

Example 5

UNIVERSITAS DIPONEGORO

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0 (1)	1 (2)	0 (3)	0 (4)
$\bar{A}B$	0 (5)	1 (6)	1 (7)	1 (8)
AB	1 (9)	1 (10)	1 (11)	0 (12)
$A\bar{B}$	0 (13)	0 (14)	1 (15)	0 (16)

$$X = \underbrace{AB\bar{C}}_{9,\,10} + \underbrace{\bar{A}\bar{C}D}_{2,\,6} + \underbrace{\bar{A}BC}_{7,\,8} + \underbrace{ACD}_{11,\,15}$$

(c)

Example 6

UNIVERSITAS DIPONEGORO

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	0	0
$\bar{A}B$	0	1	1	1
AB	0	0	0	1
$A\bar{B}$	1	1	0	1

$$X = \bar{A}\bar{C}D + \bar{A}BC + A\bar{B}\bar{C} + AC\bar{D}$$

(a)

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	0	0
$\bar{A}B$	0	1	1	1
AB	0	0	0	1
$A\bar{B}$	1	1	0	1

$$X = \bar{A}BD + BC\bar{D} + \bar{B}\bar{C}D + A\bar{B}\bar{D}$$

(b)

Filling K-Map from Expression

UNIVERSITAS DIPONEGORO

- When the desired output is presented as a Boolean expression instead of a truth table, the K map can be filled by using the following steps:
  1. Get the expression into SOP form if it is not already in that form.
  2. For each product term in the SOP expression, place a 1 in each K-map square whose label contains the same combination of input variables. Place a 0 in all other squares.

Example

- Use a K map to simplify $y = \bar{C}(\bar{A}\bar{B}\bar{D} + D) + A\bar{B}C + \bar{D}$

Solution

1. Multiply y out the first term to get $y = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{C}D + A\bar{B}C + \bar{D}$, which is now in SOP form.
2. For the $\bar{A}\bar{B}\bar{C}\bar{D}$ term, simply put a 1 in the $\bar{A}\bar{B}\bar{C}\bar{D}$ square of the K map. For the $\bar{C}D$ term, place a 1 in all squares with $\bar{C}D$ in their labels. For the $A\bar{B}C$ term, place a 1 in all squares that have an ABC in their labels. For the $\bar{D}$ term, place a 1 in all squares that have a $\bar{D}$ in their labels, that is, all squares in the leftmost and rightmost columns.

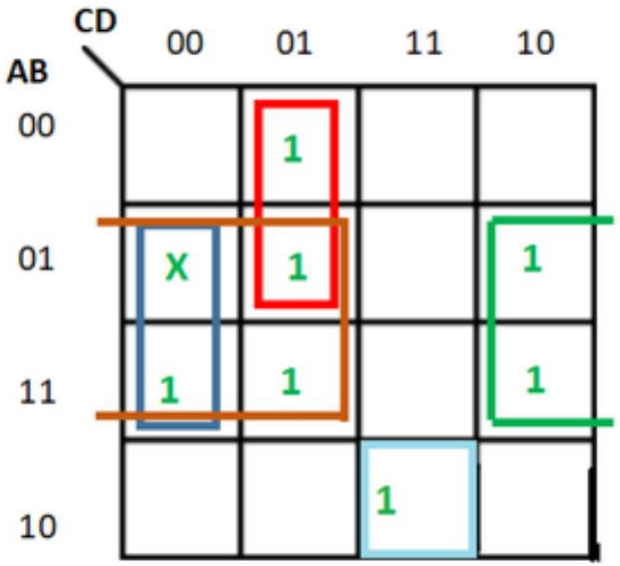
	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	1	1	0	1
$\bar{A}B$	1	1	0	1
AB	1	1	0	1
$A\bar{B}$	1	1	1	1

$y = A\bar{B} + \bar{C} + \bar{D}$

Don’t Care Condition

UNIVERSITAS DIPONEGORO

- The “Don’t Care” conditions allow us to replace the empty cell of a K-Map to form a grouping of the variables which is larger than that of forming groups without don’t care.
- While forming groups of cells, we can consider a “Don’t Care” cell as 1 or 0 or we can also ignore that cell.
- Therefore, the “Don’t Care” condition can help us to form a larger group of cells.



$$f = B\bar{C} + B\bar{D} + \bar{A}\bar{C}D + A\bar{B}CD$$

- Example: minimize the expression

$$f = \sum m(1,5,6,11,12,13) + d(4)$$